

E-Commerce Order Analytics System

Submitted by: Heeral Jain

Intern ID: CT_CSI_DE_345



Organization: Celebal Technologies

Department: Data Engineering

Internship Duration: May 2026 – July 2026

Submission Date: 12 July 2026

Acknowledgement

I would like to express my sincere gratitude to **Celebal Technologies** for providing me with the opportunity to undertake this project as part of my internship. This experience has allowed me to enhance my technical knowledge, strengthen my problem-solving skills, and gain valuable exposure to a professional work environment.

I am especially thankful to my project mentor for their continuous guidance, encouragement, and constructive feedback throughout the project. Their support and expertise played a significant role in the successful completion of this work.

I also extend my appreciation to all the members of the team at Celebal Technologies for their cooperation, support, and willingness to share their knowledge whenever needed.

Finally, I would like to thank my college, faculty members, family, and friends for their constant encouragement and support throughout the internship and the preparation of this project report.

This project has been a valuable learning experience, and I am grateful to everyone who contributed directly or indirectly to its successful completion.

Introduction

The rapid growth of e-commerce platforms has resulted in the generation of large amounts of transactional data every day. This data contains valuable information about customers, products, orders, and purchasing patterns that can help organizations make better business decisions. However, raw data collected from multiple sources often contains problems such as missing values, duplicate records, inconsistent formats, and invalid entries, which can affect the accuracy of analysis.

The **E-Commerce Order Analytics System** is developed to address these challenges by creating a complete data processing and analytics pipeline. The main purpose of this project is to collect, clean, validate, store, and analyze e-commerce order data to generate meaningful insights. The system uses Python for data generation, preprocessing, and automation, while SQLite is used for structured data storage and SQL is used for performing analytical operations

This is a data analytics project developed to transform raw e-commerce transaction data into meaningful business insights. In many organizations, data is collected from multiple operational systems, resulting in inconsistencies such as duplicate records, missing values, invalid entries, and broken relationships between datasets. These issues reduce the accuracy of business reports and decision-making.

This project focuses on building a complete analytics pipeline that generates synthetic e-commerce datasets, performs data profiling and cleaning, validates business rules, stores the cleaned data in a normalized SQLite database, and executes SQL-based analytical queries. The final system generates business reports that help organizations analyze revenue, customer behavior, product performance, and sales trends.

This project demonstrates the importance of data engineering and analytics in modern e-commerce systems. By combining data cleaning techniques, database management, and analytical methods, the system provides accurate and actionable insights that support effective decision-making and business improvement..

Project Overview

Project Title

E-Commerce Order Analytics System

Project Description

The project aims to create a reliable analytics system capable of processing raw e-commerce data and converting it into actionable business information. It follows a structured workflow beginning with data generation, followed by data cleaning, database creation, analytical query execution, and report generation.

The system ensures that only validated and high-quality data is used for analysis, improving the reliability of business intelligence reports

Objectives:-

The major objectives of this project are:

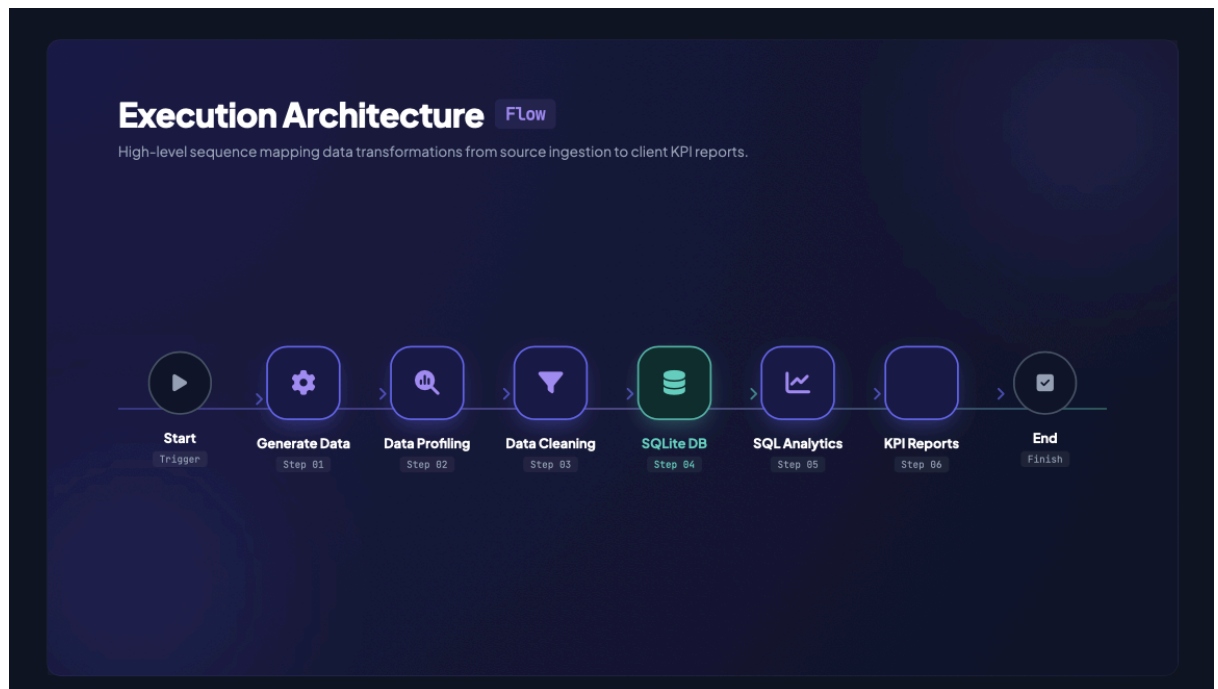
- Generate realistic synthetic e-commerce datasets.
- Identify and remove data quality issues.
- Validate business rules and database relationships.
- Store cleaned data in a normalized SQLite database.
- Perform advanced SQL-based business analytics.
- Generate business reports and KPI summaries.
- Demonstrate real-world data engineering and analytics techniques.

Workflow

The project follows a structured workflow consisting of the following stages:

1. Raw CSV files are generated using the Faker library.
2. Data profiling identifies quality issues.
3. Data cleaning removes inconsistencies and validates records.
4. Cleaned data is loaded into a normalized SQLite database.

5. SQL queries perform advanced business analysis.
6. Business reports are generated to support decision-making.



Raw CSV Files



Data Profiling



Data Cleaning & Validation



SQLite Database



SQL Analytics



Business Reports

Dataset Description

The project uses four datasets representing an e-commerce business.

Customers Dataset

Contains customer information including:

- Customer ID
- Customer Name
- Email Address
- Registration Date
- customer_type: REGULAR, PREMIUM, VIP

Products Dataset

Contains product-related information including:

- Product ID
- Product Name
- Category such as Electronics, Clothing, Home, Books
- Subcategory
- Cost Price

Orders Dataset

Contains order information including:

- Order ID
- Customer ID
- Order Date
- Order Status: PLACED, SHIPPED, DELIVERED, CANCELLED, RETURNED
- Region Code
- order_date format: YYYY-MM-DD HH:MM:SS

Order Items Dataset

Contains transactional details including:

- Item ID
- Order ID
- Product ID
- Quantity (>0)
- Unit Price
- Discount Percentage(>0 AND <100)

Data Generation

Synthetic (Fake realistic) e-commerce datasets were generated using Python's **Faker** library to simulate real-world business operations.

Faker is a Python library used to generate realistic-looking synthetic (fake) data for testing, development, and data analysis purposes. In this project,, Faker was used to create sample datasets that represent real-world business data such as customers, products, orders, and order items.

Instead of manually creating thousands of records, Faker automatically generates large volumes of realistic data with different attributes like names, emails, dates, product details, and transaction information.

```
>>from faker import Faker
```

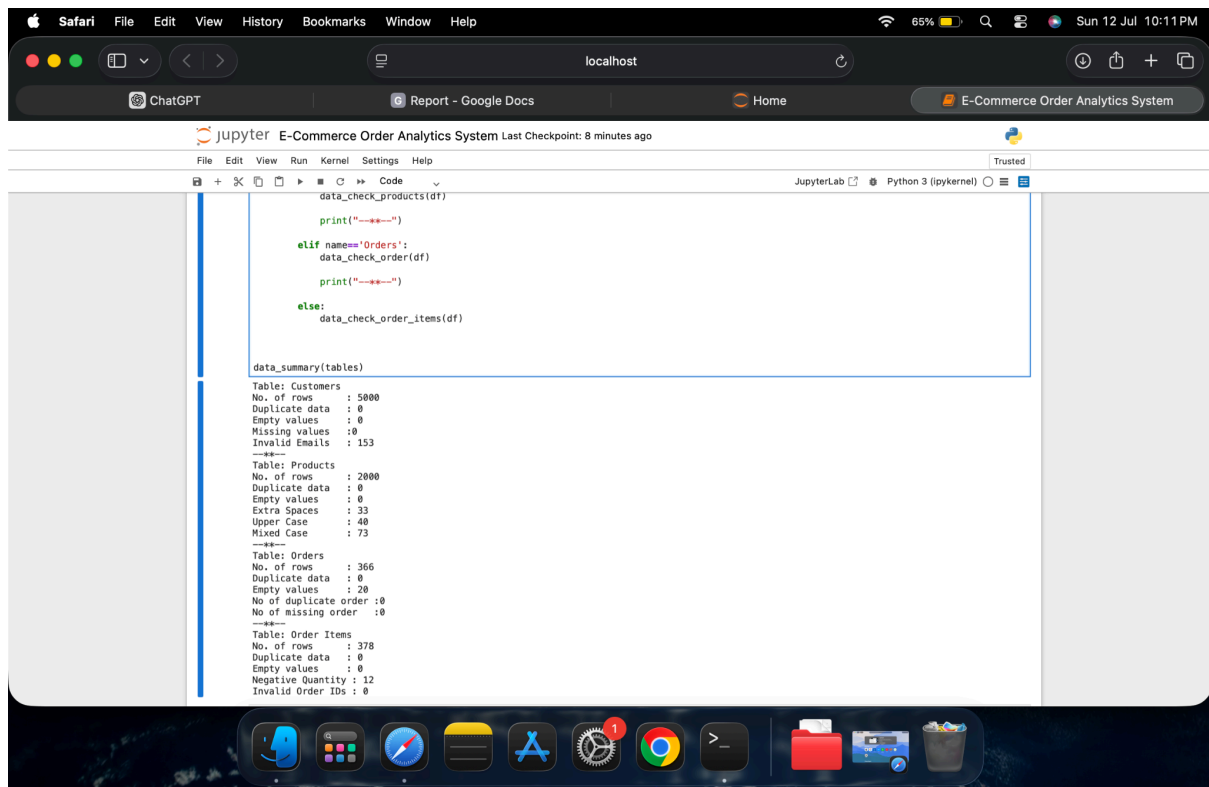
```
fake=Faker()
```

The generated datasets include customer details, product information, orders, and order items.

To simulate practical scenarios, several intentional data quality issues were introduced, such as :

- 5% of orders should have NULL customer_id
- 3% of order_items should have negative quantity
- Some orders should have order_date in wrong format (DD-MM-YYYY)
- Some product names should have extra spaces or mixed case
- 2% of emails should be invalid (missing @ or domain)

These issues provide realistic datasets for testing data cleaning and validation techniques.



Data Cleaning and Validation

Data checking and cleaning is an essential step in Data Engineering to ensure that the data used for analysis is accurate, consistent, and reliable.

Raw data collected from multiple sources often contains **errors** such as missing values, duplicate records, invalid formats, and incorrect relationships between tables. These issues can negatively affect business reports and analytical results.

After generating the datasets, several data quality checks were performed to improve accuracy and consistency.

Python libraries such as **Pandas** were used to inspect, validate, and clean the datasets.

1. Schema Validation

The structure of each dataset was verified to ensure that all required columns were present with correct data types.

```
>>>df.info()
```



```
>>Df.dtypes
```

```
>>df.describe()
```

Examples:

- Customer ID should be numeric.
- Order dates should follow a valid date format.
- Quantity and price fields should contain numerical values.

2. Missing Value Detection

Missing or null values were identified in important columns. Appropriate actions were taken based on the importance of the data.

```
>>df.isna().sum()
```

Examples:

- Missing customer names were handled.
- Missing product details were reviewed.
- Incomplete transaction records were removed or corrected.

3. Duplicate Detection

Duplicate records were identified to prevent incorrect calculations and repeated information.

```
>>df.duplicated().sum()
```

Examples:

- Duplicate customer records.
- Duplicate product entries.
- Duplicate order transactions.

4. Relationship Validation

Relationships between different datasets were checked to maintain data integrity.

Validation rules included:

- Every order_items.order_id must exist in the orders table.
- Every order_items.product_id must exist in the products table.
- Customer IDs in orders must match existing customers.

5. Business Rule Validation

Business rules were applied to detect invalid records. These validation checks significantly improve the quality of analytical reporting.

- Discount percentage must be between 0 and 100.

```
>>invalid_discount = df[(df["discount_percent"] < 0) | (df["discount_percent"] > 100)]
```

- Quantity must be greater than zero.

```
>>invalid_quantity = df[df["quantity"] <= 0]
```

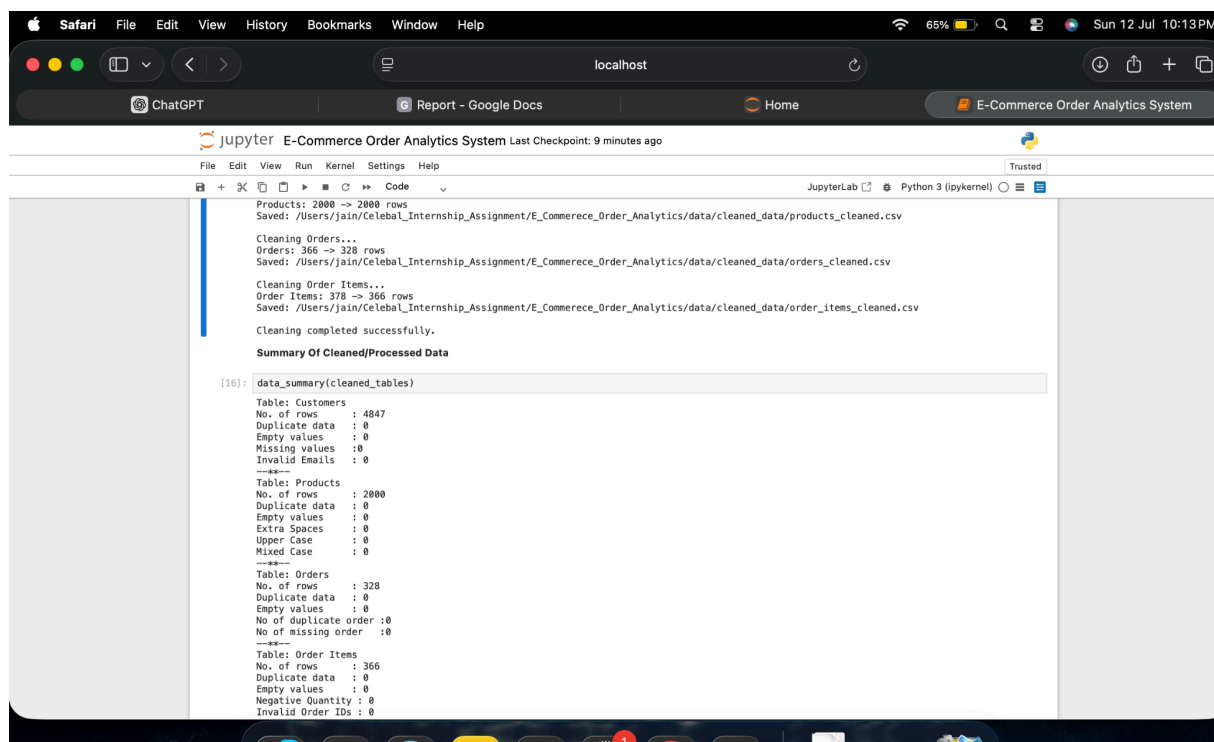
- Order date should not be a future date.

```
>>df["order_date"] = pd.to_datetime(df["order_date"])
```

- Prices should not contain negative values.

After the cleaning process, the datasets became:

- More accurate
- Consistent
- Reliable for analysis
- Suitable for database storage and reporting



```
Products: 2000 -> 2000 rows
Saved: /Users/jain/Celebal_Internship_Assignment/E_Commerce_Order_Analytics/data/cleaned_data/products_cleaned.csv

Cleaning Orders...
Orders: 366 -> 328 rows
Saved: /Users/jain/Celebal_Internship_Assignment/E_Commerce_Order_Analytics/data/cleaned_data/orders_cleaned.csv

Cleaning Order Items...
Order Items: 378 -> 366 rows
Saved: /Users/jain/Celebal_Internship_Assignment/E_Commerce_Order_Analytics/data/cleaned_data/order_items_cleaned.csv

Cleaning completed successfully.

Summary Of Cleaned/Processed Data

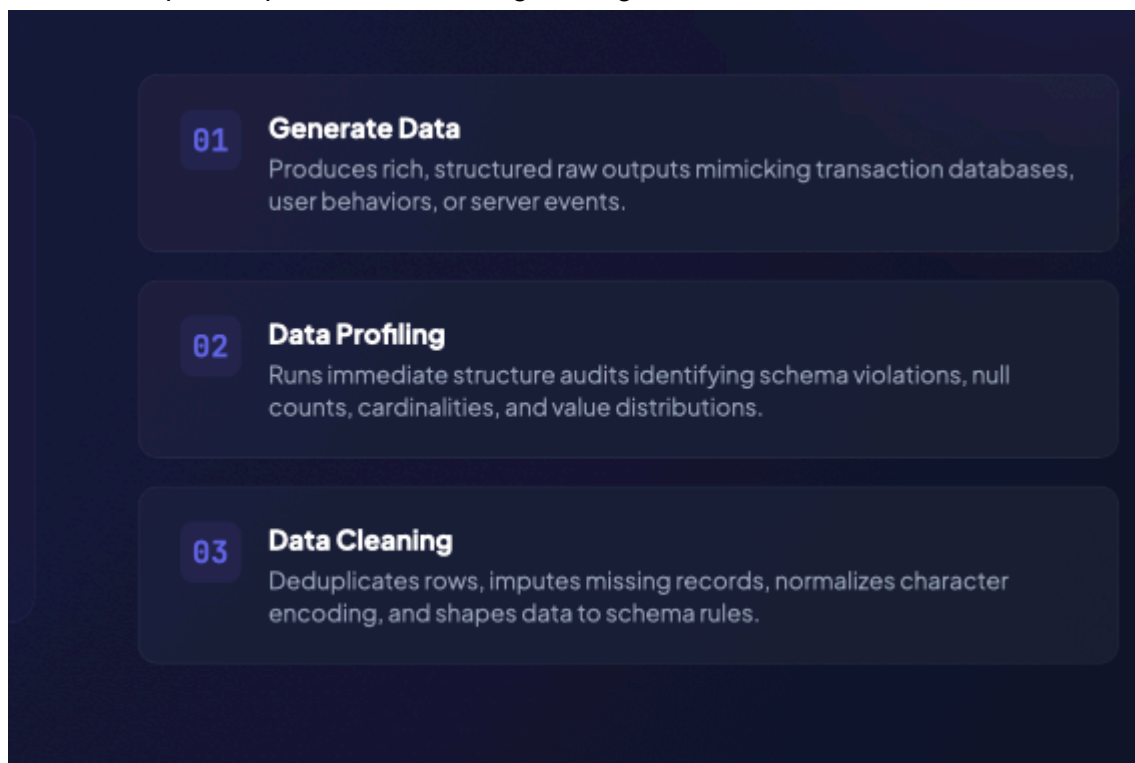
[16]: data_summary(cleaned_tables)

Table: Customers
No. of rows      : 4847
Duplicate data    : 0
Empty values     : 0
Missing values   : 0
Invalid Emails   : 0
-----
Table: Products
No. of rows      : 2000
Duplicate data    : 0
Empty values     : 0
Extra Spaces     : 0
Upper Case      : 0
Mixed Case       : 0
-----
Table: Orders
No. of rows      : 328
Duplicate data    : 0
Empty values     : 0
No of duplicate order : 0
No of missing order  : 0
-----
Table: Order Items
No. of rows      : 366
Duplicate data    : 0
Empty values     : 0
Negative Quantity : 0
Invalid Order IDs : 0
```

The cleaned data was then **loaded** into the **database** for performing SQL analytics such as revenue analysis, customer lifetime value, product performance analysis, and retention analysis.

```
>>df.to_csv("cleaned_orders.csv", index=False)
```

The most important process in Data Engineering:-



Database Design and SQL Analytics

The **E-Commerce Order Analytics System** uses a **SQLite relational database** to store and manage cleaned e-commerce data. The database stores information in structured tables .

The database is designed to organize data from multiple sources into separate tables, allowing efficient querying and analysis.

```
>>import sqlite3
connection = sqlite3.connect("ecommerce_analysis.db")
```

Basic Properties of Relational Database:-

1.Data Integrity – Since the database was created using SQLite, data reliability was maintained through preprocessing and validation before loading data into the database.

Integrity is maintained by-

- Duplicate record detection and removal.
- Missing value handling.
- Validation of order and product relationships.
- Removal of orphan order items.
- Validation of quantity and discount ranges.
- Date validation to remove invalid future orders.

These checks ensured that only clean and reliable data was stored for analysis.

2. Data Consistency -Data consistency ensures that the same data follows the same rules across the entire database.

Consistency was maintained by applying validation rules:

- discount_percent was restricted between 0 and 100.
- quantity was validated to be greater than zero.
- Date formats were standardized.
- Duplicate records were removed.
- Data types were checked before database loading.

This prevented incorrect calculations during SQL analytics.

3.Isolation (ACID Property)

Isolation ensures that multiple database operations do not interfere with each other.

Here,SQLite manages transaction isolation by controlling database access during operations.

This ensures that:

- Data loading operations do not corrupt analysis queries.
- Transactions execute independently.

4. Normalization:-- Normalization organizes data into separate tables to reduce redundancy and improve consistency.

The database was divided into:

- Customers
- Products
- Orders
- Order_Items

Instead of storing all information in a single table, related data was separated and connected logically, reducing duplicate information.

Advanced SQL queries were implemented to generate business insights including:

- Monthly Revenue Analysis
- Year-over-Year Revenue Growth
- Customer Lifetime Value (CLV)
- Category Performance Analysis
- Revenue Contribution by Customer Segment
- Top Products by Revenue
- Frequently Bought Together Analysis
- Cohort Retention Analysis
- Revenue Concentration (Pareto Analysis)

Several SQL concepts were utilized, including:

- INNER JOIN
- LEFT JOIN
- Common Table Expressions (CTEs)
- Window Functions
- Ranking Functions
- Conditional Aggregation
- Self Joins
- Cohort Analysis

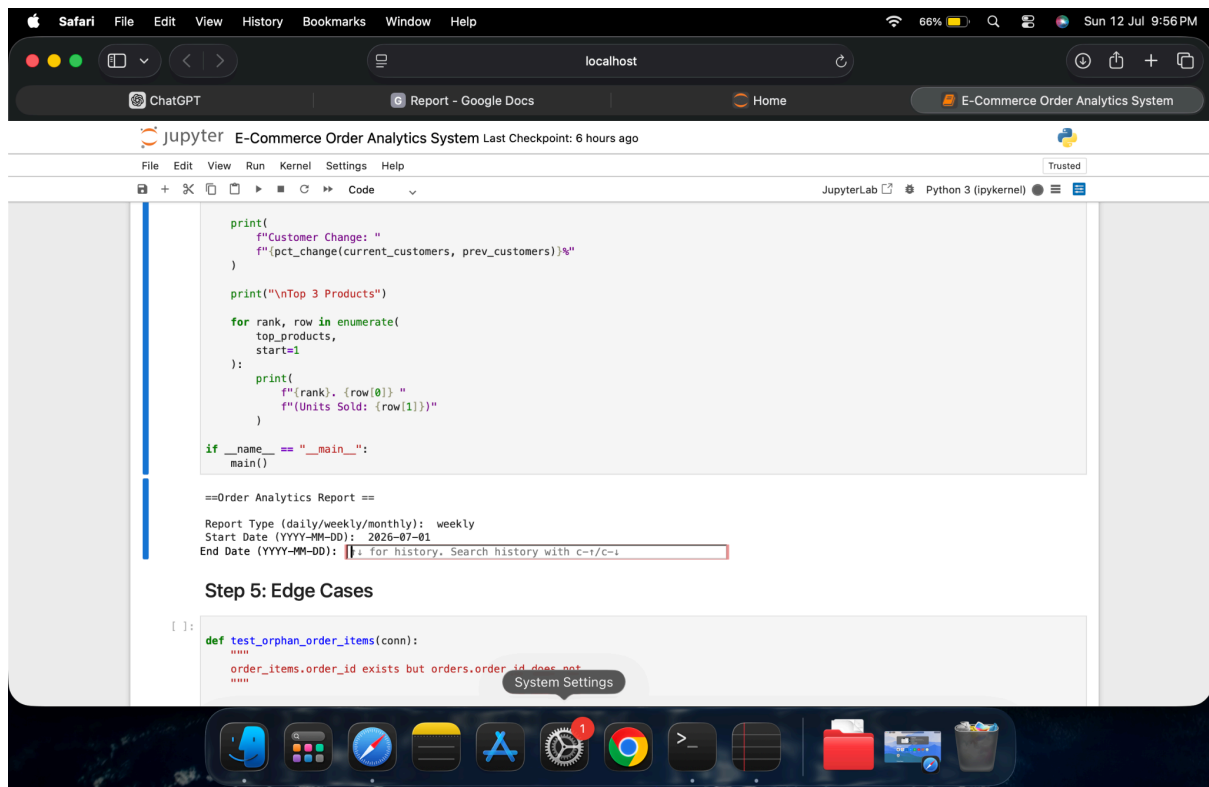
These analytical queries transform transactional data into valuable business intelligence.

Python and SQL Integration

Python and SQL integration created a complete data analytics pipeline for the E-Commerce Order Analytics System. Python handled data generation, cleaning, validation, and automation, while SQL managed structured storage and complex business analysis. This combination enabled efficient processing of e-commerce data and generation of reliable insights for decision-making.

Python was integrated with SQLite to automate report generation.

In the **E-Commerce Order Analytics System**, a **command-line reporting tool** was developed using Python to allow users to generate business reports by providing inputs directly through the terminal.



Role of Command Line in the Project

The command-line interface acts as a bridge between the user and the analytics system. It allows users to:

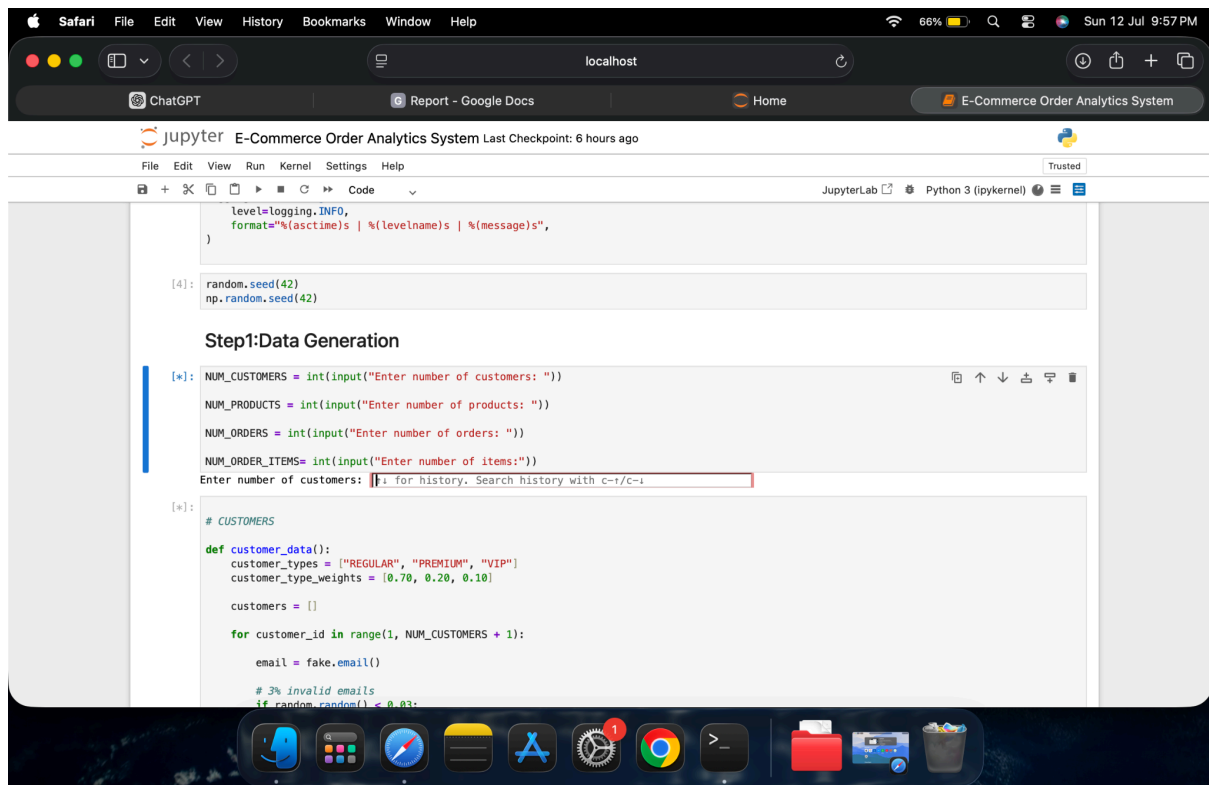
- Select the type of report they want to generate(daily/weekly/monthly).
- Enter required inputs such as date ranges.
- Execute SQL queries dynamically.
- View analytical results directly in the terminal.

Generated reports include:

- ☐ Total Orders
- ☐ Total Revenue
- ☐ Unique Customers
- ☐ Top Three Products
- ☐ Previous Period Comparison
- ☐ Growth Percentage Analysis

Advantages of Command-Line Reporting

- ❖ Simple and lightweight interface.
- ❖ Allows dynamic report generation.
- ❖ Reduces manual SQL execution.
- ❖ Automates business analysis.
- ❖ Useful for testing and backend reporting systems.



User Input

|



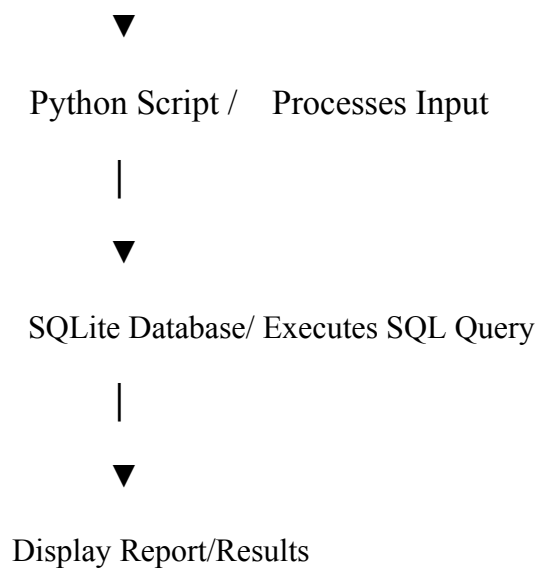
Select Report Type

|



Enter Date Range

|



```
f"(Units Sold: {row[1]})"
```

```
if __name__ == "__main__":
    main()

==Order Analytics Report ==
Report Type (daily/weekly/monthly): weekly
Start Date (YYYY-MM-DD): 2026-07-01
End Date (YYYY-MM-DD): 2026-07-07
REPORT SUMMARY
====
Period: 2026-07-01 to 2026-07-07
Report Type: Weekly

Metrics
Total Orders: 2
Revenue: 0.00
Unique Customers: 2

Previous Period Comparison
Orders Change: -50.0%
Revenue Change: -100.0%
Customer Change: -50.0%

Top 3 Products

Step 5: Edge Cases

[ ]:
def test_orphan_order_items(conn):
    """
    order_items.order_id exists but orders.order_id does not.
    """
```

Edge Case Handling:-

Edge case handling refers to identifying and managing unusual, unexpected, or extreme situations that may occur during data processing or system execution. These cases are not part of normal operations but can cause incorrect results, system failures, or inaccurate reports if they are not handled properly.

In the real-world , data comes from multiple sources and may contain errors due to manual entry, system failures, or incomplete transactions. If these issues are not handled, they can lead to:

- ❖ Incorrect revenue calculations.
- ❖ Wrong customer analysis.
- ❖ Invalid product performance reports.
- ❖ Database inconsistencies.
- ❖ System crashes during execution.

Handling edge cases improves:

- Data reliability.
- System stability.
- Report accuracy.
- Error detection.
- Overall application quality.

To improve system reliability, several validation tests were implemented.

The project detects and handles:

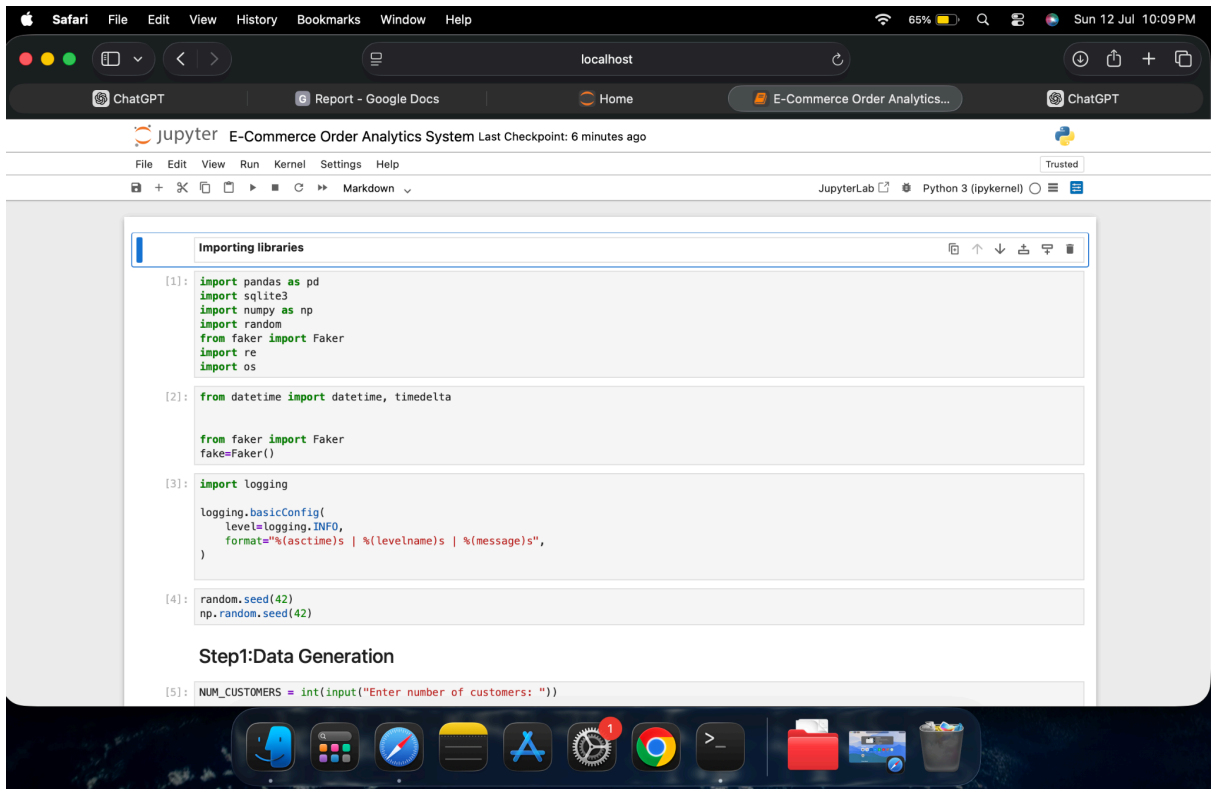
- Orphan order items
- Invalid discount percentages
- Zero quantity records
- Future-dated orders

Error handling techniques include:

- Input validation
- Database exception handling
- Invalid record logging
- Graceful failure reporting

These mechanisms improve system robustness and data integrity

Tools & Techniques-



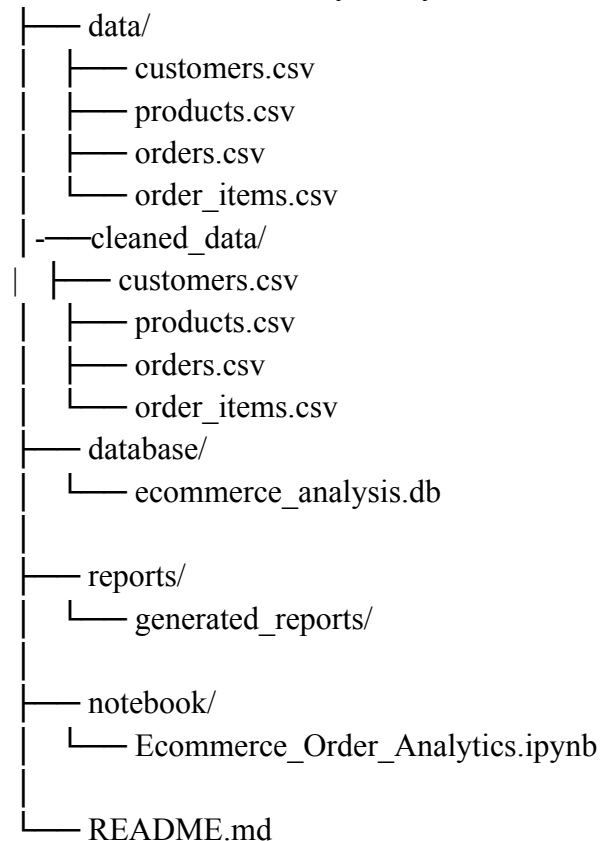
Category	Technology	General Purpose / Usage
Programming Language	Python	A high-level programming language used for application development, automation, data processing, and implementing logic.
Library	Pandas	A Python library used for data manipulation, analysis, and handling structured datasets.
Library	Faker	A Python library used for generating realistic synthetic data for testing and development purposes.

Library	Regular Expressions (re)	A Python module used for pattern matching, searching, and validating text-based data.
Library	Logging	A Python module used to track application events, errors, and execution information.
Library	OS	A Python module used for interacting with the operating system, managing files, folders, and system-related operations.
Database	SQLite	A lightweight relational database management system used for storing, managing, and retrieving structured data.
Query Language	SQL	A standard language used for creating, managing, querying, and analyzing data stored in relational databases.
Development Environment	Jupyter Notebook	An interactive development environment used for writing, executing, documenting, and visualizing code during analysis and development.

Repository Structure

The project is organized into separate folders for datasets, cleaned data, database files, reports, notebooks, and documentation. This modular structure improves project organization, maintainability, and scalability. Here is the Overview-

E-Commerce Order Analytics System/



Key Observations

The project produced several important business and technical observations:

- Duplicate records significantly affect revenue calculations.
- Data validation improves reporting accuracy.
- Referential integrity is essential for reliable database relationships.
- Window functions simplify advanced analytical calculations.
- Parameterized SQL queries improve security and maintainability.
- Early-stage data cleaning reduces errors during reporting.

KPI's

The KPIs were selected based on standard e-commerce analytics practices and the available dataset. Since the project was implemented using SQL and Python, the KPIs were chosen to evaluate sales performance, customer behavior, product performance, and operational efficiency. These metrics provide actionable insights into business performance and can be computed directly from the order data.

Key Performance Indicators (KPIs)

Business KPIs

- ❖ Total Revenue
- ❖ Total Orders
- ❖ Average Order Value
- ❖ Customer Lifetime Value
- ❖ Revenue by Category

Data Quality KPIs

- ❖ Duplicate Records Removed
- ❖ Null Values Handled
- ❖ Invalid Discounts Detected
- ❖ Future Orders Removed
- ❖ Referential Integrity Validation

Conclusion

The **E-Commerce Order Analytics System** successfully implements a complete data analytics workflow, starting from raw data generation to final business reporting. The project effectively handles common real-world data challenges such as missing values, duplicate records, invalid data, and inconsistent relationships between multiple datasets.

Through Python-based data processing and validation, the quality and reliability of the data were improved before storing it in the SQLite database. The database structure provided an organized approach for managing customer, product, order, and transaction information. SQL analytics techniques were used to generate valuable insights related to revenue performance, customer behavior, product trends, and business growth.

The integration of Python with SQL created an automated and efficient reporting system that reduces manual analysis efforts and improves decision-making. Overall, this project provided practical exposure to data cleaning, database management, SQL analytics, and software development practices. It highlights how data-driven solutions can help organizations understand their operations and make informed strategic decisions.

References

1. For the **Python Official Documentation**
<https://docs.python.org/3/>
2. **SQLite Official Documentation**
<https://www.sqlite.org/docs.html>
3. **Pandas Documentation**
<https://pandas.pydata.org/docs/>
4. **Faker Documentation**
<https://faker.readthedocs.io/>
5. **Jupyter Notebook Documentation**
<https://jupyter-notebook.readthedocs.io/>
6. **Python SQLite3 Module Documentation**
<https://docs.python.org/3/library/sqlite3.html>
7. **SQL Language Documentation (SQLite SQL Reference)**
<https://www.sqlite.org/lang.html>